Wrapping Around: Translation-Equivariant Deep Q-Learning for Toroidal Battleship

Theodora Kessling

School of Computing and Interactive Systems, Aldenmoor University
41 Harrow Quay, Port Aldenmoor, NV 89990, USA
t.kessling@cs.aldenmoor.edu

Abstract

Battleship is a compact benchmark for sequential search under uncertainty: a shooter must locate a hidden fleet using only hit, miss, and sunk feedback. We study a variant played on a torus, where the board wraps around at every edge and ships may cross the border. This removes the edges and corners that classic Battleship strategies exploit and makes the game exactly symmetric under the group $\mathbb{Z}_N \times \mathbb{Z}_N$ of board translations. We show that this symmetry can be built directly into a deep Q-network: a fully convolutional network with circular padding and no positional input is exactly translation-equivariant, needs no data augmentation, and can be applied unchanged to boards of any size. Trained with Double DQN on 5.1 million transitions in under ten minutes on a single GPU, the resulting agent clears a 10×10 toroidal board in 46.74 ± 0.28 shots on average, compared with 51.81 ± 0.27 for a hunt/target heuristic with parity that we adapted to the torus. On the same 1000 boards it needs 5.06 ± 0.39 fewer shots ($z \approx 12.8$), and it wins 66.6% (333 of 500) of head-to-head games against the heuristic. A phase decomposition shows that the gain comes mostly from the hunting phase, before any ship has been hit. We also release a toroidal Battleship testbed with a clean agent interface and evaluation harness.

1 Introduction

Battleship is one of the best-known games of search under uncertainty. Each player hides a fleet of ships on a grid, and the players take turns firing at cells of the opponent's grid, learning only whether a shot was a miss, a hit, or sank a ship. Because each shot is both an action and a query, the game is a small but non-trivial instance of the problems studied in optimal search theory (Stone 1975), Bayesian experimental design (Lindley 1956), and partially observable planning (Kaelbling, Littman, and Cassandra 1998). It has served as a benchmark for Monte-Carlo POMDP planning (Silver and Veness 2010), as a probe of human and machine question asking (Gureckis and Markant 2009; Rothe, Lake, and Gureckis 2018; Grand et al. 2026), and, more recently, as a test of imperfect-information reinforcement learning (Kubíček, Lisý, and Sandholm 2026).

Classic Battleship is played on a bounded 10×10 grid. The boundary matters: a ship can cover a corner cell in only two ways but a central cell in up to ten, so a uniform distribution over fleet layouts induces a strongly non-uniform prior over cells, and much of the folklore of good play ("shoot at the centre first", "ships hide along the edges") is about these edge effects (Berry 2011). In this paper we study *toroidal Battleship*, in which the board wraps around at every edge and ships may cross the border. Every cell of the torus is equivalent to every other: the game is invariant under the group $\mathbb{Z}_N \times \mathbb{Z}_N$ of cyclic translations, the prior probability that a cell is occupied is the same for all cells, and a good shooter must rely only on the *relative* geometry of its past observations. To our knowledge, the shooting game on a torus has not been studied before; the only wrap-around precedent we know of is a deduction puzzle (Tuller 2003).

The torus is not just a curiosity. It isolates the part of Battleship that is about inference from local evidence and removes the part that is about exploiting the boundary, and it has an exact symmetry that a learning agent can exploit. We ask two questions. First, can a deep Q-learning agent (Mnih et al. 2015) learn an effective shooting policy for this variant from self-generated experience alone? Earlier neural approaches to Battleship found reinforcement learning harder and slower than supervised hit prediction (Clementis 2013). Second, does building the torus symmetry into the network help? We answer both with a deliberately simple design: a fully convolutional Q-network whose 3×3 convolutions use *circular* padding and which receives no positional input. Such a network is exactly equivariant to board translations: shifting the board shifts the Q-values in the same way. The symmetry is therefore enforced by construction and does not have to be learned from data or imposed by augmentation, in contrast to, e.g., the symmetry augmentation of AlphaGo Zero (Silver et al. 2017). The network has no parameters tied to the board size, so the same weights apply to tori of any size.

Contributions.

1. We introduce **toroidal Battleship** as a benchmark, formalise its symmetry, and show that it makes the cell-level prior exactly uniform (Section 3). We release an open testbed with an agent interface, a vectorised training environment, and an evaluation harness that compares agents on identical boards.
2. We propose a **translation-equivariant Double DQN agent**: a circular-padded, fully convolutional dueling Q-network with per-cell outputs, trained on a discounted hit

reward with action masking (Section 4). We prove that it is exactly $\mathbb{Z}_N \times \mathbb{Z}_N$-equivariant, matching the symmetry of the optimal action-value function, and verify this numerically.

3. We give a **paired evaluation** against random and hunt/target baselines (Section 5). On identical boards the learned agent needs 5.06 fewer shots than a hunt/target heuristic with parity (about 10%), is better on 659 of 1000 boards and worse on 314, and wins 333 of 500 (66.6%) head-to-head games. Training took under ten minutes on one GPU. We further analyse where the savings come from, what the agent's Q-maps look like, and how the network behaves on board sizes it was never trained on.

2 Related Work

Battleship as search and inference. The standard strategies for the shooting game are random firing, *hunt/target* (fire randomly until a hit, then probe its neighbours), hunt/target with *parity* (hunt only on one colour of a checkerboard, since every ship covers both colours), and *probability density*, which fires at the cell covered by the most fleet layouts consistent with the observations (Berry 2011). On the classic board these are reported to need roughly 65 and 42 shots for parity hunt/target and probability density, respectively (Berry 2011); because our board has no edges and our ships may touch, these numbers are not directly comparable to ours. Audinot, Bonnet, and Viennot (2014) look for the shooting strategy that minimises the expected number of shots against uniformly random layouts, which is exactly our solo setting; they compute optima on small grids, show that greedy maximum-probability play is not always optimal, and bound the optimum on 10×10. Crombez, da Fonseca, and Gerard (2020) analyse the worst case of the target phase, i.e., how many misses are needed to sink a ship of known shape after a first hit. Sevenster (2004) shows that the related Battleship *puzzle* is NP-complete, which indicates that exact inference over consistent layouts is hard in general. Monte-Carlo comparisons of attacking and placement strategies are given by Didych and Malinetskii (2017), and Moltchanova et al. (2024) analyse diagonal and probability-ordered hunting in a related "Warships" game. Naslausky and Figueiredo (2025) estimate hit probabilities by sampling consistent boards with Monte Carlo tree search. Battleship has also been a testbed for information seeking in cognitive science (Gureckis and Markant 2009; Rothe, Lake, and Gureckis 2017, 2018) and, recently, for language-model agents (Grand et al. 2026; Tajwar et al. 2025).

Learning-based Battleship agents. Clementis (2013) cast Battleship as a POMDP and trained a neural network to predict hit probabilities, finding reinforcement learning harder and slower to converge than supervised learning. Kancko (2020) formulated the game as an MDP and studied MCTS. POMCP (Silver and Veness 2010) used 10×10 Battleship with a reward of $+1$ per hit as one of its three benchmark POMDPs; we use the same per-hit reward. Cauwet and Teytaud (2018) optimised strategies for partially observable games, including a Battleship variant, with evolutionary methods. Closest in spirit to our match setting, Kubíček, Lisý, and Sandholm (2026) use two-player Battleship as a main benchmark for test-time policy-gradient refinement of a game-theoretic blueprint and evaluate against random and checkerboard hunt/target opponents (with rules that forbid touching ships). None of these works uses a symmetry-preserving value network, and none considers a toroidal board.

Deep Q-learning. Our agent builds on Q-learning (Watkins and Dayan 1992) with deep networks (Mnih et al. 2015), experience replay (Lin 1992), a target network, Double DQN targets (van Hasselt, Guez, and Silver 2016), and a dueling head (Wang et al. 2016). Further extensions such as prioritised replay (Schaul et al. 2016), distributional value functions (Bellemare, Dabney, and Munos 2017), and their combination in Rainbow (Hessel et al. 2018) are orthogonal to our contribution. Under partial observability, recurrent Q-networks are a common choice (Hausknecht and Stone 2015); in Battleship they are unnecessary because the shooter's board view, together with the list of ships still afloat, is a sufficient statistic of the observation history (Section 4.1).

Symmetry and equivariance. Convolutional weight sharing makes networks equivariant to translations, and group-equivariant CNNs extend this to rotations and reflections (Cohen and Welling 2016). In reinforcement learning, MDP homomorphic networks (van der Pol et al. 2020) and equivariant DQNs for grid games such as Snake and Pacman (Mondal, Nair, and Siddiqi 2020) build environment symmetries into policies and value functions and report better sample efficiency. On a bounded image, zero padding breaks exact translation equivariance and leaks absolute position into the features; circular padding is the exact match for periodic domains (Alguacil et al. 2021) and is the natural choice for toroidal grids such as the Game of Life (Springer and Kenyon 2020). Our per-cell Q-head follows the fully convolutional design of Long, Shelhamer, and Darrell (2015), which is also used for per-intersection move prediction in Go (Clark and Storkey 2015; Silver et al. 2017). KataGo (Wu 2019) and GNN-based AlphaZero variants (Ben-Assayag and El-Yaniv 2021) exploit architectures without size-specific parameters to play several board sizes, which motivates our transfer experiment.

Other query games. Battleship belongs to a family of games in which an agent chooses queries to reduce uncertainty about a hidden state, alongside Mastermind (Knuth 1976) and Minesweeper (Kaye 2000). Deep Q-learning on Minesweeper (Mehta 2021) faces a similar per-cell action space and partial observability.

3 Toroidal Battleship

Rules. We describe the rules exactly as implemented in our testbed. The board is an $N \times N$ torus $\mathbb{Z}_N \times \mathbb{Z}_N$, with $N = 10$ by default. The fleet consists of ships of lengths $5, 4, 3, 3, 2$, i.e., 17 ship cells out of 100. A ship of length ℓ

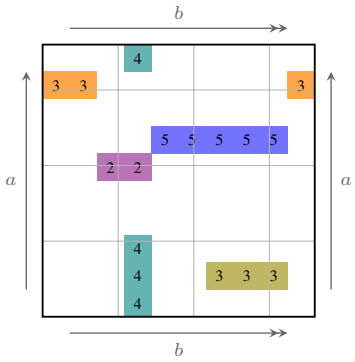


Figure 1: A random fleet on the 10×10 torus (ship cells are labelled by ship length). Opposite edges marked a and b are identified, so several ships wrap across an edge. Ships may touch but not overlap.

is placed horizontally or vertically starting at any cell and occupies ℓ consecutive cells, wrapping modulo N (Figure 1). Ships may touch but may not overlap. Fleets are placed by the *random placement* procedure used by all our agents: ships are placed longest first, each uniformly at random among the $2N^2$ placements (for $\ell \geq 2$) that do not overlap the ships already placed, restarting in the rare case of a dead end. Players take turns firing at a cell of the opponent's board; each shot is answered with *miss*, *hit*, *sunk* (together with the length of the ship), or *repeat* if the cell had already been shot, in which case the turn is wasted. When a ship is sunk, its cells are revealed to the shooter. Each player may fire at most $2N^2$ shots; if neither player has sunk the opposing fleet by then, the game is a draw. (The limit only matters for agents that waste shots; none of our agents ever does.)

What the torus changes. Let τ_v denote translation of the board by $v \in \mathbb{Z}_N^2$. Every rule above commutes with τ_v: a translated legal layout is legal, a translated shot on a translated layout produces the same feedback, and the placement procedure assigns the same probability to a layout and its translate (at every step it chooses uniformly from a candidate set that is translated along with the layout). This has a simple but important consequence.

Proposition 1. *Under any translation-invariant layout distribution on the $N \times N$ torus, in particular under random placement, every cell is occupied with probability exactly $17/N^2$ (0.17 for $N = 10$) before the first shot.*

Proof. Let $p(x)$ be the probability that cell x is occupied. Invariance gives $p(x) = p(x + v)$ for all v, so p is constant, and $\sum_x p(x) = 17$ because every layout occupies exactly 17 cells. □

On a bounded board, in contrast, a ship of length 5 can cover a corner cell in 2 ways but a central cell in 10, and good heuristics exploit this. On the torus there are no edges and no corners, every cell is covered by exactly 2ℓ placements of a ship of length ℓ, and the first shot carries no positional information at all. Hand-written heuristics must also be adapted: "extend a line of hits" has to wrap around, and a run of hits can in principle form a closed ring. Parity hunting still works when N is even, because the checkerboard colouring $(r + c) \bmod 2$ is well defined on $\mathbb{Z}_N \times \mathbb{Z}_N$ and every ship of length ≥ 2 covers both colours; when N is odd the colouring does not close up across the seam and parity is no longer sound.

The game is also invariant under the dihedral group D_4 of rotations and reflections of the square torus (rotating a layout by 90° swaps horizontal and vertical placements, which are equally likely). We build in only the translations and examine the remaining symmetry empirically in Section 5.7.

Evaluation settings. We use two settings. In the *solo* game a single shooter fires at a fixed random fleet until every ship is sunk, and the score is the number of shots to clear the board (lower is better). This is the single-agent problem studied by Audinot, Bonnet, and Viennot (2014), the MDP on which we train, and the cleanest measure of targeting strength. In the two-player *match* game both players shoot in alternation at each other's random fleets, and the first to sink the opposing fleet wins; the starting player alternates from game to game. Since all our agents place ships with the same random procedure, the match outcome depends only on shooting skill, and the starting player wins whenever both players need the same number of shots.

4 Approach

4.1 MDP Formulation

We train on the solo game. Although the fleet is hidden, the problem can be cast as a fully observed MDP over *information states*. The state s_t is the shooter's view of the opponent's board, where every cell is *unknown*, *miss*, *hit* (on a ship not yet sunk), or *sunk*, together with the multiset of lengths of the ships still afloat. Since the layout distribution is fixed and the feedback is deterministic given the layout, the posterior over layouts depends on the history only through the set of shots fired and their outcomes, which this view records exactly when sunk cells are revealed. The view is therefore a sufficient statistic, and no recurrent memory is needed (cf. Hausknecht and Stone 2015).

The state is encoded as $4 + 6$ planes of size $N \times N$: four one-hot planes for the cell states, and six constant planes, where plane k holds the number of remaining ships of length $k + 1$ (lengths above 6 share the last plane). The constant planes tell the network, for example, that the length-2 ship is still afloat, which governs how small a gap between misses can hide a ship. No plane encodes position.

Actions are cells. Cells that have already been shot are masked, both when acting and when computing bootstrap targets, so the agent can never waste a shot. The reward is $r_t = 1$ if the shot hits and 0 otherwise, and the episode ends when the fleet is cleared.

Why a discounted hit reward. Our objective is to minimise the number of shots T to clear the board. Every episode contains exactly 17 hits, so the undiscounted return is the constant 17 and carries no learning signal. With discount $\gamma < 1$ the return is $G = \sum_{j=1}^{17} \gamma^{t_j}$, where $t_1 < \cdots <$

$t_{17} = T - 1$ are the (zero-based) times of the hits, which rewards getting every hit as early as possible. G is a surrogate rather than an exact transformation of T: if one policy makes each of its hits no later than another, it has both a larger G and a smaller T, but in general the two objectives can trade off (e.g., an early hit bought with a later final hit). An exact alternative is a reward of -1 per shot; we keep the hit reward of POMCP (Silver and Veness 2010) because it is dense and bounded and because, with $\gamma = 0.9$, the horizon of about ten shots matches the timescale of the target phase. We select checkpoints by T, not by G (Section 4.3).

4.2 Translation-Equivariant Q-Network

The Q-network (Figure 2) is a stack of six 3×3 convolutional layers with 64 channels, ReLU activations, and *circular* padding, followed by a dueling head (Wang et al. 2016). The advantage stream is a 1×1 convolution that produces one value $A(s, a)$ per cell a; the value stream applies global average pooling to the last hidden layer and a linear layer to obtain $V(s)$; and $Q(s, a) = V(s)+A(s, a)-\frac{1}{N^2}\sum_{a'} A(s, a')$. The six layers give a receptive field of 13×13, which on a 10×10 torus already covers the whole board around every cell. The network has 190,594 parameters.

Proposition 2. *Let Q_θ be the network above and let τ_v act on states by translating all planes and on actions by $a \mapsto a+v$. Then $Q_\theta(\tau_v s, a+v) = Q_\theta(s, a)$ for all s, a, v and all parameters θ. Moreover, the optimal action-value function satisfies the same identity, $Q^*(\tau_v s, a+v) = Q^*(s, a)$.*

Proof sketch. A convolution with circular padding on $\mathbb{Z}_N^2$ is a cyclic cross-correlation, which commutes with cyclic shifts; pointwise ReLU and 1×1 convolutions commute with shifts; the constant ship-count planes are shift-invariant; and global average pooling and the mean over advantages are shift-invariant, so V and $\overline{A}$ are unchanged. For Q^*: translation maps the layout distribution to itself (Section 3), and the transition and reward functions commute with τ_v, so τ_v is an MDP automorphism and the Bellman optimality operator commutes with it; its unique fixed point is therefore equivariant (cf. van der Pol et al. 2020). □

Proposition 2 has three practical consequences. First, restricting the hypothesis class to equivariant functions loses nothing, since Q^* lies in the equivariant class, while every training transition is effectively shared across all N^2 translations. Second, the policy is coherent with the symmetry of the problem: on the empty board all Q-values are identical, so the first shot is uniformly random (ties are broken at random), which is optimal by Proposition 1. Third, no weight depends on N, so the same network can be run on any board size; we test this in Section 5.8. A zero-padded network would instead see the image border as a spurious edge and would have to learn to ignore it.

4.3 Training

We train with Double DQN (van Hasselt, Guez, and Silver 2016) (Algorithm 1). Transitions are generated by a vectorised NumPy implementation of the solo game that steps 256 boards in parallel, samples fleets with exactly the same distribution as the game engine, and resets each board as soon as it is cleared. Exploration is ϵ-greedy over legal cells, with ϵ annealed linearly from 1.0 to 0.01 over the first 25% of training. Transitions are stored in a replay buffer of capacity 10^6, which is kept on the GPU as compact integer codes and decoded into planes on the fly. Learning starts after 20,000 transitions; afterwards we perform two gradient updates per environment step, each on a minibatch of 512 transitions, with the Huber loss, gradient-norm clipping at 10, and Adam with a learning rate of $3 \cdot 10^{-4}$ that decays linearly to 10% of its initial value. The target network is synchronised every 1000 updates. Table 1 lists all hyperparameters; we did not tune them systematically.

Algorithm 1 Double DQN training on vectorised boards

Initialise Q_θ, target $Q_{\bar\theta} \leftarrow Q_\theta$, replay buffer $\mathcal{D}$
Reset $B = 256$ boards with random fleets
for step $= 1$ **to** 20,000 **do**
  $\epsilon \leftarrow$ linear schedule from 1.0 to 0.01 over first 25%
  For each board: $a \leftarrow$ random legal cell w.p. ϵ, else $\arg\max_{a \text{ legal}} Q_\theta(s, a)$
  Fire all B shots; store $(s, a, r, s', \text{done})$ in $\mathcal{D}$; reset cleared boards
  if $|\mathcal{D}| \geq 20{,}000$ **then**
    for 2 gradient updates **do**
      Sample a minibatch of 512 transitions from $\mathcal{D}$
      $a^* \leftarrow \arg\max_{a' \text{ legal in } s'} Q_\theta(s', a')$
      $y \leftarrow r + \gamma(1 - \text{done})\, Q_{\bar\theta}(s', a^*)$
      Adam step on Huber$(Q_\theta(s, a) - y)$; clip grad. norm at 10
      Every 1000 updates: $\bar\theta \leftarrow \theta$
    end for
  end if
  Every 1000 steps: greedy evaluation on 1000 held-out boards; keep the best θ by mean shots to clear
end for

The full run consists of 20,000 vectorised steps, i.e., about 5.1M transitions from about 103k complete games and 40k gradient updates, and took 9.4 minutes on a single GPU. Every 1000 steps we evaluate the greedy policy on a fixed set of 1000 held-out boards (drawn from a different seed than all test boards) and keep the checkpoint with the lowest mean number of shots to clear.

Deployment. At play time the agent encodes its view, evaluates the network, and fires at the unexplored cell with the highest Q-value, breaking ties uniformly at random. Inference is a pure NumPy reimplementation of the forward pass (circular padding via wrap-around indexing and an im2col convolution), so the trained agent has no deep-learning dependency. Like all agents in this paper it places its own ships with the random placement procedure; we study only the shooting side.

4.4 Baselines

Random. Fires at a uniformly random unexplored cell.

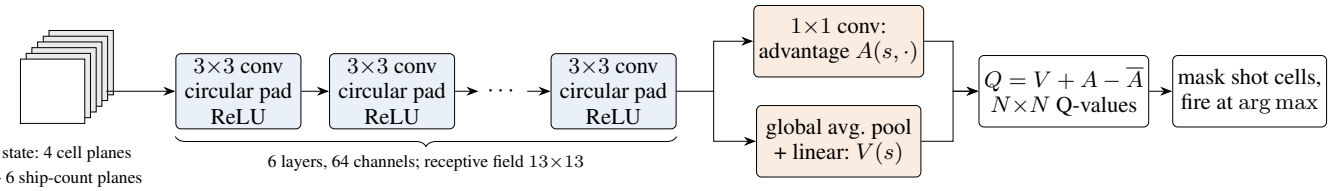


Figure 2: The Q-network. All convolutions use circular padding and there is no positional input, so translating the input board by v translates the output Q-map by v (Proposition 2). The value stream uses global average pooling and is translation-invariant. No parameter depends on the board size.

Hyperparameter	Value
Conv. layers / channels / kernel	6 / 64 / 3×3, circular
Head	dueling (avg. pool value)
Discount γ	0.9
Optimiser, learning rate	Adam, $3 \cdot 10^{-4} \rightarrow 3 \cdot 10^{-5}$
Batch size, updates per step	512, 2
Replay capacity, warm-up	10^6, $2 \cdot 10^4$ transitions
Target sync	every 1000 updates
ϵ schedule	$1.0 \rightarrow 0.01$ over 25% of steps
Parallel boards, env. steps	256, 20,000
Loss, grad. clipping	Huber, norm 10
Evaluation	every 1000 steps, 1000 boards
Seed	0

Table 1: Training configuration of the DQN agent.

Hunt/target with parity ("hunt"). The classic heuristic (Berry 2011), adapted to the torus. In *target* mode, i.e., whenever there is a hit on a ship that is not yet sunk, it fires at an unknown cell that extends a run of two or more adjacent hits along its axis, if there is one, and otherwise at an unknown orthogonal neighbour of a hit; all neighbourhoods and runs wrap around the edges. In *hunt* mode it fires at a random unexplored cell of one checkerboard colour, chosen at random at the start of each game, and falls back to any unexplored cell when that colour is exhausted (and on odd boards, where parity is not sound). Because sunk cells are revealed, the agent automatically returns to hunting once all hit ships are sunk.

The obvious stronger baseline, a probability-density agent that counts consistent layouts, is not yet part of our study; we return to this in the discussion.

5 Experiments

5.1 Setup

All experiments use the default rules: a 10×10 torus, fleet $5, 4, 3, 3, 2$, sunk cells revealed, and a limit of 200 shots per player. The evaluated network is the checkpoint selected during training (Section 4.3); none of the test boards was used for training or checkpoint selection. In the *solo* benchmark each agent clears the *same* 1000 random fleets (master seed 0), which permits paired comparisons. In the *match* benchmark every pair of agents plays 500 games in a round robin, alternating the starting player. We report the number of shots to clear (mean $\pm$ standard error, median, range), accuracy (hits per shot, averaged over games), wasted repeat shots, win rates, and paired differences on identical boards. The complete evaluation (3000 solo games and 1500 matches) runs in under two minutes on a CPU.

Agent	Mean $\pm$ SEM	Median	Min	Max	Accuracy
Random	95.20 ± 0.16	97	68	100	0.179
Hunt/target	51.81 ± 0.27	53	24	68	0.339
DQN (ours)	$\mathbf{46.74 \pm 0.28}$	**46**	**23**	73	**0.378**

Table 2: Solo benchmark: shots to clear a 10×10 toroidal board (lower is better) on the same 1000 random fleets. No agent ever wastes a shot on a repeat.

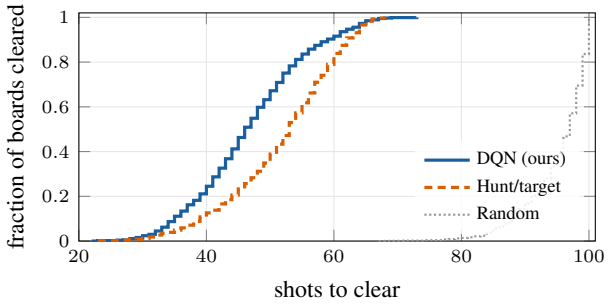


Figure 3: Empirical distribution of shots to clear on the 1000 solo boards. The DQN curve lies to the left of the hunt/target curve over almost the whole range.

5.2 Solo Results

Table 2 shows the solo results. Random firing needs 95.2 shots on average; its accuracy of 0.179 is close to the base rate of 0.17 from Proposition 1 (slightly higher only because the game ends at the last hit). Hunt/target with parity nearly halves this to 51.8 shots. The DQN agent needs 46.7 shots, about 5 fewer, and has the highest accuracy: 37.8% of its shots are hits. No agent ever fires a repeat shot: the DQN agent cannot because illegal cells are masked, and the heuristics only choose unexplored cells.

Paired comparison. Since all agents face identical boards, we can compare them board by board. The DQN agent needs 5.06 ± 0.39 fewer shots than hunt/target ($z \approx 12.8$). It is better on 659 of the 1000 boards, ties on 27, and is worse on 314; a sign test on the 973 non-tied boards gives

Agent	Win rate vs. Random	Hunt	DQN	Overall
Random	–	0.0%	0.0%	0.0%
Hunt/target	100.0%	–	33.4%	66.7%
DQN (ours)	100.0%	**66.6%**	–	**83.3%**

Table 3: Match benchmark: win rates of the row agent in head-to-head games (500 games per pairing, alternating first player, no draws).

$z \approx 11.1$. Both agents beat random firing on every single board, by 43.40 ± 0.32 (hunt) and 48.46 ± 0.32 (DQN) shots.

Distribution. Figure 3 shows the full distribution. The DQN curve is shifted to the left of the hunt/target curve by roughly five shots across most of the range: its median is 46 rather than 53, and its 90th percentile is 59 rather than 62. The two agents have similar spread (standard deviation 8.86 vs. 8.69). The tails differ in an instructive way. The DQN agent needs 60 or more shots on only 97 boards, compared with 210 for the heuristic, but its single worst board needs 73 shots, five more than the heuristic's worst case of 68, and it needs 68 or more shots on 5 boards, against 3. A parity hunter can never spend more than 50 shots hunting, because it only fires at one colour of the checkerboard and every ship covers both colours. The learned policy has no such hard guarantee, and it does not follow strict parity (see below). Its extreme tail is therefore slightly heavier, even though its typical and bad-case performance (90th and 99th percentiles of 59 and 67 shots, against 62 and 66) is about as good as or better than the heuristic's.

5.3 Match Results

Table 3 shows the round robin. Both learned and heuristic shooters win all 500 of their games against random firing, and no game is drawn. Head to head, the DQN agent beats hunt/target in 333 of 500 games (66.6%, 95% Wilson interval [62.4%, 70.6%]), i.e., it wins two games for every one it loses. The overall win rates are 83.3% for DQN, 66.7% for hunt/target, and 0% for random. When the DQN agent wins, it needs 44.86 ± 0.29 shots, compared with 50.24 ± 0.36 for the heuristic's wins.

The match result is what the solo distributions predict. If both players clear independent random boards with their solo shot-count distributions, and ties go to the starting player (half of the games), the predicted DQN win rate is 66.9%, within the confidence interval of the observed 66.6%. Since our agents ignore the opponent's progress, the two-player game adds nothing beyond the race between the two solo distributions; exploiting the opponent's state is left to future work.

5.4 Training Dynamics

Figure 4 shows the greedy evaluation over training. Already after the first evaluation (256k transitions, about 2.9k games, $\epsilon = 0.8$), the greedy policy needs 49.6 shots and is thus already better than the hunt/target heuristic (51.8 on the test boards). Performance briefly gets worse towards the end of the ϵ schedule (51.4 shots at 1.28M transitions), presumably because the replay buffer suddenly fills with near-greedy data and the value targets shift. It then recovers quickly and settles between 46.2 and 46.9 shots for the second half of training; the evaluations from 2.3M transitions onwards span only 0.7 shots, less than three standard errors of a single evaluation. The best checkpoint (46.23 on the held-out boards, at 4.35M transitions) scores 46.74 on the independent test boards. The small gap is consistent with the mild optimism of picking the best of 20 noisy evaluations, and it shows that checkpoint selection did not overfit the held-out boards in any meaningful way. Training is therefore stable, and any of the late checkpoints would have given nearly the same result.

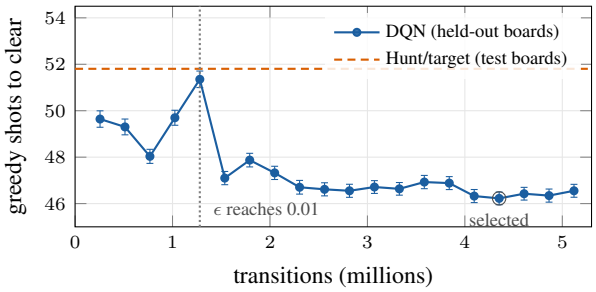


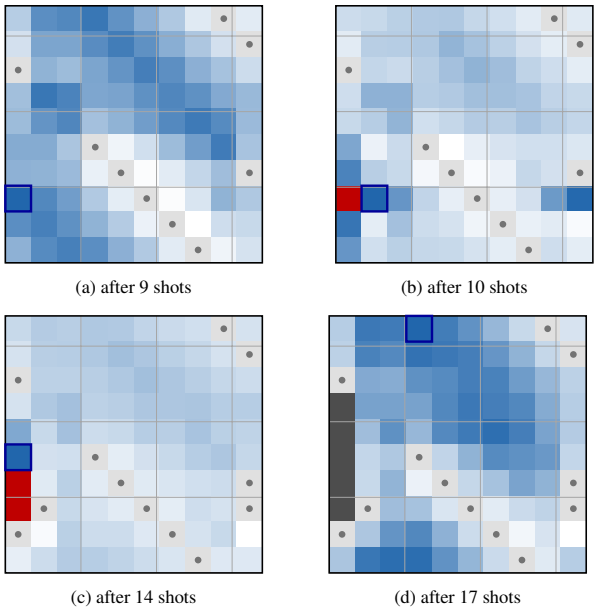
Figure 4: Greedy evaluation during training (mean ± SEM over 1000 held-out boards, every 1000 vectorised steps). The circled point is the checkpoint used in all experiments.

	Hunt/target	DQN	Saving
Hunt-phase shots	33.27 ± 0.28	28.49 ± 0.30	4.78
of which misses	28.65	23.85	4.80
ships found by hunting	4.62	4.64	
Target-phase shots	18.53 ± 0.07	18.25 ± 0.06	0.28
of which misses	6.15	5.90	0.25
Total shots to clear	51.81 ± 0.27	46.74 ± 0.28	5.06
Shots to first hit	5.45 ± 0.14	5.02 ± 0.12	0.43
Majority-colour hunt shots	100%	64%	

Table 4: Decomposition of shots on the 1000 solo boards (mean ± SEM per board). A shot is a *target* shot if, when it is fired, some hit belongs to a ship that is not yet sunk, and a *hunt* shot otherwise.

5.5 Where Do the Savings Come From?

To find out where the five saved shots come from, we split every game into *hunt* shots, fired while no unsunk hit is known, and *target* shots, fired while at least one hit belongs to a ship that is still afloat (Table 4). Almost the entire gain comes from hunting: the DQN agent spends 4.78 fewer shots in the hunt phase but only 0.28 fewer in the target phase. Both agents find about 4.6 of the 5 ships by hunting (the rest are found by accident while targeting an adjacent ship), so the learned agent does not hunt differently in *what* it finds but in *how quickly* it finds it. The target phase, in contrast,



(a) after 9 shots (b) after 10 shots

(c) after 14 shots (d) after 17 shots

Figure 5: Q-values of the DQN agent at four moments of one solo game (board seed 0, cleared in 42 shots). Unexplored cells are shaded from white (lowest Q-value on the board) to blue (highest); grey cells with a dot are misses, red cells are hits on unsunk ships, black cells are sunk ships. The outlined cell is the agent's next shot. Each map is normalised separately.

is close to saturated for both agents: about 18.3–18.5 target shots for the remaining ≈ 12.4 ship cells, i.e., about 6 misses or 1.2 per ship.

The hunting difference is not explained by parity. The heuristic fires every hunt shot on its chosen checkerboard colour, whereas on average only 64% of the DQN agent's hunt shots fall on the majority colour of the game. Colour-blind shooting over a hunt phase of 28 shots would give about 57% (binomial approximation), and strict parity gives 100%, so the learned agent has at most a weak colour preference. A plausible reason is that, once some misses and sunk ships are on the board, the cell most likely to hold a ship is determined by the lengths of the gaps between known cells and by which ships are still afloat, not by a fixed colouring. This is exactly the information that a probability-density strategy uses (Berry 2011) and that our ship-count planes provide. The DQN agent also finds its first ship faster (5.02 vs. 5.45 shots). Before the first hit the only evidence is the set of misses, so this gain must come from placing early shots such that each miss rules out many placements and raises the hit probability of the remaining cells.

5.6 Learned Hunting and Targeting

Figure 5 shows the agent's Q-values at four moments of one game. (a) After nine misses, the Q-map is smooth and roughly anti-correlated with the misses: cells inside or next to the band of misses get the lowest values, and the highest values lie in the largest unexplored regions, where the most ship placements are still possible. (b) The next shot hits the cell at the left edge of the board. Immediately, the four torus neighbours of the hit get the four highest Q-values on the board (normalised values 100, 98, 90, and 81). The second-highest is the neighbour *across the left edge*, in the last column: the network treats the seam exactly like any other cell boundary, as Proposition 2 guarantees. (c) After a second hit directly above the first and a miss below it, the network extends the vertical line upwards. The next cell on the line has the maximal value, the one after it the second-highest (57), and every other cell is at most 41. (d) Once the ship of length 5 is sunk and revealed, the agent returns to hunting, and the map again favours large regions far from misses and from the sunk ship. Without being given any rule, the agent thus reproduces the hunt/target structure, including wrap-around targeting and line extension, and it hunts in a way that resembles a density map rather than a fixed checkerboard.

5.7 Symmetry: Exact and Learned

Translations (built in). We verify Proposition 2 numerically on 200 states sampled from the agent's own games, each with a random translation. The largest absolute difference between $Q(\tau_v s)$ and the translated $Q(s)$ is $1.4 \cdot 10^{-6}$, whereas the Q-values themselves reach 5.07; this difference is float32 rounding. On the empty board the 100 Q-values differ by at most $2.4 \cdot 10^{-7}$. An extended check on 2271 states from 50 games finds that the greedy shot is always mapped to the translated greedy shot.

Rotations and reflections (not built in). The game is also invariant under D_4, but our network is not constrained to respect it. On the same 2271 states, the median largest deviation between $Q(gs)$ and $gQ(s)$ is 35–43% of the range of Q-values on the board, depending on the transformation g. The greedy shot of the transformed state agrees with the transformed greedy shot in only 39–48% of states. The disagreement is concentrated in the hunt phase (22–40% agreement, versus 59–72% in the target phase), where many cells are nearly equally good. To test whether this inconsistency costs performance, we symmetrise the network at test time, $\bar{Q}(s) = \frac{1}{8}\sum_{g \in D_4} g^{-1} Q(gs)$, which makes the policy equivariant to the full symmetry group $\mathbb{Z}_N^2 \rtimes D_4$ of the square torus without any retraining. On the 1000 test boards the symmetrised agent needs 46.06 ± 0.29 shots, against 46.39 ± 0.28 for the plain network in the same run (paired difference 0.33 ± 0.38, better on 501 boards and worse on 468), which is not a significant improvement. This suggests that the network's violations of D_4 mostly concern near-equivalent hunting moves.

(In this run the plain network scores 46.39 rather than the 46.74 of Table 2, because it was evaluated with a different number of linear-algebra threads. On sparse boards several cells have nearly identical Q-values, and floating-point summation order can then change which one is chosen, after which the games diverge. The two numbers are about one standard error apart, which indicates how much a single 1000-board evaluation of the same policy can vary.)

N	Random	Hunt/target	DQN	Saving	Better
8	61.2	38.32 ± 0.24	$\mathbf{34.39} \pm 0.22$	10.2%	344 : 127
$9^{\dagger}$	77.2	50.87 ± 0.52	$\mathbf{39.91} \pm 0.31$	21.5%	388 : 99
10^{*}	95.2	51.81 ± 0.27	$\mathbf{46.74} \pm 0.28$	9.8%	659 : 314
$11^{\dagger}$	115.2	70.22 ± 0.83	$\mathbf{52.88} \pm 0.49$	24.7%	387 : 106
12	137.9	68.91 ± 0.56	$\mathbf{60.13} \pm 0.63$	12.7%	336 : 156
$15^{\dagger}$	213.2	122.53 ± 1.67	$\mathbf{90.44} \pm 1.01$	26.2%	369 : 131

Table 5: Zero-shot transfer: shots to clear on $N \times N$ tori with the same fleet, using the network trained only on $N = 10$ (*; 1000 boards, from Table 2). Other sizes use 500 boards, identical across agents. "Saving" is the paired reduction relative to hunt/target; "Better" counts boards on which DQN needs fewer : more shots. †Odd sizes, where the heuristic cannot use parity.

5.8 Zero-Shot Transfer to Other Board Sizes

Because the network has no size-dependent parameters, we can run the agent trained on 10×10 on other tori without any retraining (Table 5). It beats the hunt/target heuristic on every size we tried, from 8×8 to 15×15. On even sizes, where the heuristic uses parity, the saving is 10–13%, similar to the 9.8% on the training size. On odd sizes the checkerboard does not close up, the heuristic loses parity, and the learned agent's advantage grows to 21–26%. The result on 15×15 is notable: the ship density drops from 17% to 7.6%, the board has more than twice as many cells as any board seen in training, and the 13×13 receptive field no longer covers the whole torus. Yet the agent still needs 32 fewer shots than the heuristic. We have not compared against networks trained on these sizes, so we cannot say how much performance transfer costs. The results nevertheless show that the policy the network learns is a local, size-independent targeting rule rather than a memorised 10×10 pattern, as one would expect from a fully convolutional, translation-equivariant design (Wu 2019; Ben-Assayag and El-Yaniv 2021).

6 Discussion and Limitations

What the results do and do not show. Our evidence that reinforcement learning works well for Battleship is solid within its scope: the comparison is paired, the effect is more than twelve standard errors from zero, and the solo distributions accurately predict the match outcome. The scope, however, is narrow. All results come from a single training run (seed 0) with untuned hyperparameters, so we cannot yet quantify the variance across seeds, although the flat late phase of the training curve suggests that it is modest. We have not run ablations, so we cannot attribute the gain to individual design choices. In particular, the claim that circular padding *helps* rather than merely being correct remains to be tested against a zero-padded network with and without positional planes. The planned ablations are zero versus circular padding, removing the ship-count planes, removing the dueling head, varying γ, and training directly on other board sizes and fleets.

Missing baseline. The most important open comparison is with a probability-density agent that fires at the cell covered by the most consistent layouts (Berry 2011; Naslausky and Figueiredo 2025), and with the optimal-play bounds of Audinot, Bonnet, and Viennot (2014). Hunt/target with parity is the standard heuristic, but on the classic board probability density is reported to be considerably stronger, and we are not aware of a learned agent that has been shown to beat it. We therefore do not claim that our agent is near-optimal; we claim that it clearly beats the standard heuristic and that it learns, without being told, a hunting strategy that departs from parity and is more efficient. On the torus a density agent is easy to implement exactly for small boards and by sampling for larger ones, and translation equivariance would let it share computation across shifts.

Rule differences. Our rules allow touching ships, reveal sunk cells, and, of course, wrap around. Shot counts are thus not comparable to numbers reported for the classic game (Berry 2011) or for rule sets that forbid touching (Kubíček, Lisý, and Sandholm 2026). Revealing sunk cells matters for our sufficient-statistic argument; with hidden sunk cells the view is no longer sufficient, because it does not say which hits belong to the sunk ship, and a recurrent or history-based network would be the principled choice.

Placement. All agents place their fleets uniformly at random, so the defender side of the game is not studied. A learned or adversarial placement strategy would change the layout distribution and break Proposition 1 unless it, too, is translation-invariant, which is a natural constraint to study in the two-player game.

7 Conclusion and Future Work

We introduced toroidal Battleship, a variant whose wrap-around board removes all edge effects and makes the game exactly symmetric under board translations, and we showed how to build this symmetry into a deep Q-network. A small fully convolutional network with circular padding, trained with Double DQN for under ten minutes on one GPU, clears a 10×10 toroidal board in 46.7 shots on average, about 10% fewer than a hunt/target heuristic with parity on the same boards, and wins two out of three head-to-head games against it. The savings come mainly from hunting: the network learns to place its blind shots where they are most informative given the misses so far and the ships still afloat. Because no parameter depends on the board size, the same network also plays well on tori it has never seen.

Future work follows directly from the limitations. (1) Compare against a probability-density or exact Bayesian agent and against the theoretical bounds of Audinot, Bonnet, and Viennot (2014). (2) Run the ablations above with several seeds. (3) Learn ship *placement*, adversarially or by self-play. (4) Train directly on the two-player game, where the agent can condition on the opponent's progress and, for example, take more risk when behind (cf. Kubíček, Lisý, and Sandholm 2026). (5) Train one network across board sizes, fleets, and rule variants, including hidden sunk cells and other topologies such as the cylinder or the Klein bottle, where only a subgroup of the translations survives. (6) Extend the equivariance to the full symmetry group

$\mathbb{Z}_N^2 \rtimes D_4$ with group-equivariant convolutions (Cohen and Welling 2016; Mondal, Nair, and Siddiqi 2020). (7) Use stronger value learning, such as distributional or Rainbow-style DQN (Bellemare, Dabney, and Munos 2017; Hessel et al. 2018), or combine the learned Q-values with search such as POMCP (Silver and Veness 2010). (8) Interpret the learned hunting pattern and compare it with exact posterior occupancy probabilities.

References

Alguacil, A.; Gonçalves Pinto, W.; Bauerheim, M.; Jacob, M. C.; and Moreau, S. 2021. Effects of Boundary Conditions in Fully Convolutional Networks for Learning Spatio-Temporal Dynamics. In *Machine Learning and Knowledge Discovery in Databases (ECML-PKDD 2021), Applied Data Science Track*.

Audinot, M.; Bonnet, F.; and Viennot, S. 2014. Optimal Strategies against a Random Opponent in Battleship. In *Proceedings of the 19th Game Programming Workshop (GPW 2014)*, 67–74. Information Processing Society of Japan.

Bellemare, M. G.; Dabney, W.; and Munos, R. 2017. A Distributional Perspective on Reinforcement Learning. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*.

Ben-Assayag, S.; and El-Yaniv, R. 2021. Train on Small, Play the Large: Scaling Up Board Games with AlphaZero and GNN. ArXiv:2107.08387.

Berry, N. 2011. Battleship. DataGenetics blog, http://www.datagenetics.com/blog/december32011/.

Cauwet, M.-L.; and Teytaud, O. 2018. Surprising Strategies Obtained by Stochastic Optimization in Partially Observable Games. In *2018 IEEE Congress on Evolutionary Computation (CEC)*. ArXiv:1807.01877.

Clark, C.; and Storkey, A. 2015. Training Deep Convolutional Neural Networks to Play Go. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*.

Clementis, L. 2013. Supervised and Reinforcement Learning in Neural Network Based Approach to the Battleship Game Strategy. In *Nostradamus 2013: Prediction, Modeling and Analysis of Complex Systems*, volume 210 of *Advances in Intelligent Systems and Computing*, 191–200. Springer.

Cohen, T. S.; and Welling, M. 2016. Group Equivariant Convolutional Networks. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48 of *PMLR*, 2990–2999.

Crombez, L.; da Fonseca, G. D.; and Gerard, Y. 2020. Efficient Algorithms for Battleship. In *10th International Conference on Fun with Algorithms (FUN 2020)*, Leibniz International Proceedings in Informatics (LIPIcs). ArXiv:2004.07354.

Didych, Y. O.; and Malinetskii, G. G. 2017. Statistical Analysis of "Battleship" Strategies. *Herald of the Bauman Moscow State Technical University, Series Natural Sciences*, 4(73): 122–135.

Grand, G.; Pepe, V.; Andreas, J.; and Tenenbaum, J. B. 2026. Shoot First, Ask Questions Later? Building Rational Agents That Explore and Act Like People. In *International Conference on Learning Representations (ICLR)*.

Gureckis, T. M.; and Markant, D. B. 2009. Active Learning Strategies in a Spatial Concept Learning Game. In *Proceedings of the 31st Annual Conference of the Cognitive Science Society*, 3145–3150.

Hausknecht, M.; and Stone, P. 2015. Deep Recurrent Q-Learning for Partially Observable MDPs. In *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*.

Hessel, M.; Modayil, J.; van Hasselt, H.; Schaul, T.; Ostrovski, G.; Dabney, W.; Horgan, D.; Piot, B.; Azar, M.; and Silver, D. 2018. Rainbow: Combining Improvements in Deep Reinforcement Learning. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*.

Kaelbling, L. P.; Littman, M. L.; and Cassandra, A. R. 1998. Planning and Acting in Partially Observable Stochastic Domains. *Artificial Intelligence*, 101(1–2): 99–134.

Kancko, T. 2020. *Reinforcement Learning for the Game of Battleship*. Bachelor's thesis, Faculty of Informatics, Masaryk University.

Kaye, R. 2000. Minesweeper is NP-Complete. *The Mathematical Intelligencer*, 22(2): 9–15.

Knuth, D. E. 1976. The Computer as Master Mind. *Journal of Recreational Mathematics*, 9(1): 1–6.

Kubíček, O.; Lisý, V.; and Sandholm, T. 2026. Test-Time Reinforcement Learning in Imperfect Information Games. ArXiv:2608.30635.

Lin, L.-J. 1992. Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching. *Machine Learning*, 8(3–4): 293–321.

Lindley, D. V. 1956. On a Measure of the Information Provided by an Experiment. *The Annals of Mathematical Statistics*, 27(4): 986–1005.

Long, J.; Shelhamer, E.; and Darrell, T. 2015. Fully Convolutional Networks for Semantic Segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Mehta, A. 2021. Reinforcement Learning for Constraint Satisfaction Game Agents (15-Puzzle, Minesweeper, 2048, and Sudoku). *Canadian Science Fair Journal*, 4(1). ArXiv:2102.06019.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. 2015. Human-Level Control through Deep Reinforcement Learning. *Nature*, 518(7540): 529–533.

Moltchanova, E.; Moyers-González, M.; Van de Voorde, G.; Voloch, J. F.; and Wacker, P. 2024. How to Survive the Squid Games Using Probability Theory. ArXiv:2409.05263.

Mondal, A. K.; Nair, P.; and Siddiqi, K. 2020. Group Equivariant Deep Reinforcement Learning. ArXiv:2007.03437.

Naslausky, E.; and Figueiredo, D. R. 2025. Sampling Battleships Boards for Hit Estimation Using Monte Carlo Tree Search. In *Anais do XXII Encontro Nacional de Inteligência Artificial e Computacional (ENIAC 2025)*, 640–651.

Rothe, A.; Lake, B. M.; and Gureckis, T. M. 2017. Question Asking as Program Generation. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*.

Rothe, A.; Lake, B. M.; and Gureckis, T. M. 2018. Do People Ask Good Questions? *Computational Brain & Behavior*, 1(1): 69–89.

Schaul, T.; Quan, J.; Antonoglou, I.; and Silver, D. 2016. Prioritized Experience Replay. In *International Conference on Learning Representations (ICLR)*.

Sevenster, M. 2004. Battleships as Decision Problem. *ICGA Journal*, 27(3): 142–149.

Silver, D.; Schrittwieser, J.; Simonyan, K.; Antonoglou, I.; Huang, A.; Guez, A.; Hubert, T.; Baker, L.; Lai, M.; Bolton, A.; et al. 2017. Mastering the Game of Go without Human Knowledge. *Nature*, 550: 354–359.

Silver, D.; and Veness, J. 2010. Monte-Carlo Planning in Large POMDPs. In *Advances in Neural Information Processing Systems 23 (NIPS 2010)*, 2164–2172.

Springer, J. M.; and Kenyon, G. T. 2020. It's Hard for Neural Networks to Learn the Game of Life. ArXiv:2009.01398.

Stone, L. D. 1975. *Theory of Optimal Search*. Academic Press.

Tajwar, F.; Jiang, Y.; Thankaraj, A.; Rahman, S. S.; Kolter, J. Z.; Schneider, J.; and Salakhutdinov, R. 2025. Training a Generally Curious Agent. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*.

Tuller, D. 2003. Wraparound Battleships. Logic puzzle, *GAMES World of Puzzles*.

van der Pol, E.; Worrall, D. E.; van Hoof, H.; Oliehoek, F. A.; and Welling, M. 2020. MDP Homomorphic Networks: Group Symmetries in Reinforcement Learning. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*.

van Hasselt, H.; Guez, A.; and Silver, D. 2016. Deep Reinforcement Learning with Double Q-Learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, 2094–2100.

Wang, Z.; Schaul, T.; Hessel, M.; van Hasselt, H.; Lanctot, M.; and de Freitas, N. 2016. Dueling Network Architectures for Deep Reinforcement Learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48 of *PMLR*, 1995–2003.

Watkins, C. J. C. H.; and Dayan, P. 1992. Q-Learning. *Machine Learning*, 8(3–4): 279–292.

Wu, D. J. 2019. Accelerating Self-Play Learning in Go. ArXiv:1902.10565.